

Lowest Common Ancestor (LCA)

Company:- Amazon

Round :- In-person interviews at Amazon office.

Year:- 2026

Difficulty:- Medium to Hard

Topic:- Binary Trees, Recursion, Tree Traversal, Divide and Conquer, and Ancestor Concepts.

Source:- Desi Qna

Problem Description :-

QUESTION : LOWEST COMMON ANCESTOR (LCA)

Given a binary tree and two nodes p and q, find their Lowest Common Ancestor (LCA).

The Lowest Common Ancestor of two nodes is the lowest node in the tree that has both p and q as descendants.

Input Format :-

1. A Binary Tree
2. Two nodes p and q

Output Format :-

Return the node that represents the Lowest Common Ancestor of p and q.

Sample Input:-

```
      3
     / \
    5   1
   / \ / \
  6  2 0 8
   / \
  7   4

p = 5
q = 1
```

Sample output :- 3

Explanation :- Node 3 is the Lowest Common Ancestor (LCA) of nodes 5 and 1 because 3 is the first node from which we can reach both of them. From 3, we can go to the left and reach 5, or go to the right and reach 1. Since there is no other node below 3 that connects both 5 and 1, 3 is their closest common ancestor.

Approach to solve this problem :-

1) Start from the root of the tree.

The root is 3, so we first check node 3. We need to find whether 5 and 1 are somewhere below this node.

2) If the current node is null, return null.

Suppose while searching we reach a position where there is no node, such as beyond node 6. Since there is nothing there to search, we return null.

3) If the current node is either p or q, return the current node.

Here, $p = 5$ and $q = 1$. When our search reaches node 5, we have found one of the required nodes, so we return 5.

4) Recursively search in the left and right subtrees.

From node 3, we search both sides. The left side contains 5, while the right side contains 1, so both searches eventually find one target node.

5) If both left and right recursive calls return a non-null node, the current node is the LCA.

At node 3, the left search returns 5 and the right search returns 1. Since both sides contain our required nodes, 3 is their Lowest Common Ancestor.

6) Otherwise, return whichever side contains a target node.

For example, if we were checking node 5 and only its left side contained one of our target nodes, we would return that result. This helps pass the information back to the parent node until the LCA is found.

Java Solution :-

```
class Solution {  
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
        if (root == null || root == p || root == q) {  
            return root;  
        }  
  
        TreeNode left = lowestCommonAncestor(root.left, p, q);  
        TreeNode right = lowestCommonAncestor(root.right, p, q);  
  
        if (left != null && right != null) {  
            return root;  
        }  
  
        return left != null ? left : right;  
    }  
}
```

Solution:-

1. Check the current node:
If the node is null, or it is p or q, return that node because we have either reached the end or found one of the required nodes.
2. Search the left side:
Recursively search the left subtree to find whether p or q is present there.
3. Search the right side:
Recursively search the right subtree in the same way.
4. Check both results:
If the left side finds one node and the right side finds the other, the current node is their Lowest Common Ancestor.
5. Return the found node:
If only one side contains a target node, return that side's result so the information can move back up through the recursion.

COMPLEXITY ANALYSIS :-

Time Complexity: $O(n)$

Space Complexity: $O(h)$, where h is the height of the tree.